

What is Web Storage



What is **Web Storage**?

Web Storage is an API that provides a mechanism by which browsers can store key/value pairs locally within the user's browser.

It does this in a much more intuitive fashion than using cookies.

It provides two mechanisms for storing data:

localStorage

sessionStorage

localStorage

No expiration date
Same across tabs

sessionStorage

Cleared on tab close
Per tab only

Both store data as key/value pairs

Local Storage vs Session Storage

Both are part of the Web Storage API and share the same simple methods, but they differ in how long the data lives.



Local Storage

Stores data for the current origin with no expiration date.

- Data stays until explicitly removed
- Persists after the browser is closed
- Shared across all tabs of the same origin



Session Storage

Stores data for one session, lost when the browser tab is closed.

- Data lives only for that tab's session
- Cleared automatically on tab close
- Not shared between different tabs



Key Concepts

The building blocks you will use with the Web Storage API



Key / Value

Data is stored as simple string pairs, like a key and its value.

```
setItem(key, value)
```



localStorage

Persists with no expiration date, until removed.

```
localStorage.setItem(...)
```



sessionStorage

Lives only for the current tab's session.

```
sessionStorage.setItem(...)
```



Storage Object

Each origin gets its own separate storage area.

```
window.localStorage
```



Example: localStorage



localStorage keeps data for the current origin with no expiration date, even after the browser is closed.



save.js

```
// Save data
localStorage.setItem("username", "farnood");

// Read data
const user = localStorage.getItem("username");
console.log(user); // "farnood"
```



remove.js

```
// Remove one item
localStorage.removeItem("username");

// Remove everything
localStorage.clear();

// Still there after refresh / restart
// until explicitly removed
```



localStorage data survives page refreshes and even closing the browser.



Example: sessionStorage

sessionStorage works the same way, but the data is lost once that browser tab is closed.



save.js

```
// Save data for this tab only
sessionStorage.setItem("step", "2");

// Read data
const step = sessionStorage.getItem("step");
console.log(step); // "2"
```



lifecycle.js

```
// Opening a new tab to the same site
// does NOT share this data

// Closing this tab clears it
sessionStorage.removeItem("step");
sessionStorage.clear();
```



sessionStorage is perfect for data that should not outlive one tab's session.



Web Storage vs Cookies

Feature	Cookies	Web Storage
Storage size	About 4 KB	5 MB or more
Sent to server	Yes, with every request	No, stays in the browser
Expiration	Set manually	localStorage: none / sessionStorage: tab close
API style	String-based, manual parsing	Simple setItem / getItem
Access	Client and server	Client (browser) only

Use cookies when the server needs the data too. Use Web Storage for browser-only data.



Key Takeaways

1

Browser-based key/value storage

Web Storage stores key/value pairs locally within the user's browser.

2

More intuitive than cookies

Simple setItem / getItem methods, with no manual parsing needed.

3

localStorage

Stores data for the current origin with no expiration date.

4

sessionStorage

Stores data for one session; lost when the browser tab is closed.

